

Student Performance Prediction — End-to-End ML Project

6-Week AI Internship Final Assignment

Pipeline: Problem Definition → Data Collection → Data Cleaning → Data Analysis → Model Development → Training → Prediction → Evaluation → Conclusion

1. Problem Definition

Project Title: Student Performance Prediction

Problem Statement: Educators struggle to identify, early in a term, which students are at risk of underperforming in final exams. Given measurable student attributes (study hours, attendance, prior performance, sleep, extracurricular involvement, parental involvement, internet access), can we predict a student's final exam score?

Objective: Build a regression model that predicts a student's final exam score (0-100) from behavioural and demographic features, and evaluate how well it generalizes to unseen students.

Why AI/ML is suitable: The relationship between study habits and exam outcomes is influenced by many interacting factors that are hard to encode with fixed rules. ML models can learn these non-trivial, non-linear interactions directly from data and generalize to new students.

Expected outcome: A trained regression model that predicts final exam score with reasonably low error (RMSE) and a high R^2 score, plus insights into which factors matter most.

Real-world application: Schools/EdTech platforms can use such a model for early-warning systems, personalized tutoring recommendations, and resource allocation for at-risk students.

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
sns.set_theme(style="whitegrid", palette="deep")
CHARTS = "/home/claude/project/charts"
```

2. Data Collection

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
df = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/student_performance_raw.csv")
print("Shape:", df.shape)
df.head()
```

Shape: (620, 12)

	student_id	gender	age	hours_studied	attendance_percent	previous_score	sleep_hours	extracurricular_activities	parental_involvement	internet_access	weekly_class_hours	final_exam_score
0	1315	Male	15	3.2	52.8	42.0	6.5	No	No	No	12	85.0
1	1163	Male	16	7.6	67.4	72.3	5.6	No	No	No	10	78.0
2	1202	Female	15	4.8	73.6	58.2	7.5	No	No	No	11	82.0
3	1290	Male	17	8.2	100.0	69.9	6.9	Yes	Yes	Yes	13	90.0
4	1024	Female	17	3.8	81.7	74.4	5.5	No	No	No	9	75.0

Next steps: [Generate code with df](#) [New interactive sheet](#)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 620 entries, 0 to 619
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   student_id                           620 non-null    int64
1   gender                               620 non-null    object
2   age                                  620 non-null    int64
3   hours_studied                        620 non-null    float64
4   attendance_percent                   595 non-null    float64
5   previous_score                       610 non-null    float64
6   sleep_hours                         601 non-null    float64
7   extracurricular_activities           620 non-null    object
8   parental_involvement                 608 non-null    object
9   internet_access                     608 non-null    object
10  weekly_class_hours                   620 non-null    int64
11  final_exam_score                    620 non-null    float64
dtypes: float64(5), int64(3), object(4)
memory usage: 58.3+ KB
```

df.dtypes

```

student_id      int64
gender          object
age             int64
hours_studied   float64
attendance_percent float64
previous_score  float64
sleep_hours     float64
extracurricular_activities object
parental_involvement object
internet_access object
weekly_class_hours int64
final_exam_score float64

dtype: object
```

3. Data Cleaning / Preprocessing

```
print("=== BEFORE CLEANING ===")
print("Rows, Columns:", df.shape)
print("\nMissing values per column:\n", df.isnull().sum())
print("\nDuplicate rows:", df.duplicated().sum())
print("\nUnique gender values:", df["gender"].unique())
before_shape = df.shape
before_missing = df.isnull().sum().sum()
before_dupes = df.duplicated().sum()
```

```

=== BEFORE CLEANING ===
Rows, Columns: (620, 12)

Missing values per column:
  student_id      0
  gender          0
  age             0
  hours_studied   0
  attendance_percent 25
  previous_score  10
  sleep_hours     19
  extracurricular_activities 0
  parental_involvement 12
  internet_access 12
  weekly_class_hours 0
  final_exam_score 0
  dtype: int64

Duplicate rows: 20

Unique gender values: ['Male' 'Female' 'male' 'female']

```

```

df_clean = df.copy()

# Standardize inconsistent categorical text
df_clean["gender"] = df_clean["gender"].str.strip().str.title()

# Remove duplicate rows
df_clean = df_clean.drop_duplicates()

# Handle missing values:
# - Numeric columns -> median imputation (robust to outliers)
# - Categorical columns -> mode imputation
num_cols_with_na = ["attendance_percent", "sleep_hours", "previous_score"]
cat_cols_with_na = ["parental_involvement", "internet_access"]
for col in num_cols_with_na:
    df_clean[col] = df_clean[col].fillna(df_clean[col].median())
for col in cat_cols_with_na:
    df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

# Drop non-predictive identifier column
df_clean = df_clean.drop(columns=["student_id"])

```

```

print("=== AFTER CLEANING ===")
print("Rows, Columns:", df_clean.shape)
print("\nMissing values per column:\n", df_clean.isnull().sum())
print("\nDuplicate rows:", df_clean.duplicated().sum())
print("\nUnique gender values:", df_clean["gender"].unique())
after_shape = df_clean.shape
after_missing = df_clean.isnull().sum().sum()
after_dupes = df_clean.duplicated().sum()
cleaning_summary = pd.DataFrame({
    "Metric": ["Rows", "Columns", "Total missing values", "Duplicate rows"],
    "Before": [before_shape[0], before_shape[1], before_missing, before_dupes],
    "After": [after_shape[0], after_shape[1], after_missing, after_dupes],})
os.makedirs('/home/claude/project/data', exist_ok=True)
cleaning_summary.to_csv("/home/claude/project/data/cleaning_summary.csv", index=False)
print(cleaning_summary)

```

```

=== AFTER CLEANING ===
Rows, Columns: (600, 11)

Missing values per column:
  gender      0
  age         0
  hours_studied 0
  attendance_percent 0
  previous_score 0
  sleep_hours   0
  extracurricular_activities 0
  parental_involvement 0
  internet_access 0
  weekly_class_hours 0
  final_exam_score 0
  dtype: int64

Duplicate rows: 0

Unique gender values: ['Male' 'Female']

```



	Metric	Before	After
0	Rows	620	600
1	Columns	12	11
2	Total missing values	78	0
3	Duplicate rows	20	0

Feature & Target Selection

Target variable: `final_exam_score` (continuous, regression problem)

Features: all other columns — `gender`, `age`, `hours_studied`, `attendance_percent`, `previous_score`, `sleep_hours`, `extracurricular_activities`, `parental_involvement`, `internet_access`, `weekly_class_hours`.

```
df_clean.to_csv("/content/drive/MyDrive/Colab Notebooks/student_performance_raw.csv", index=False)
df_clean.describe(include="all").T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
gender	600	2	Male	312	NaN	NaN	NaN	NaN	NaN	NaN	NaN
age	600.0	NaN	NaN	NaN	16.501667	1.121946	15.0	15.75	16.0	18.0	18.0
hours_studied	600.0	NaN	NaN	NaN	5.021	2.189364	0.0	3.6	5.05	6.4	11.2
attendance_percent	600.0	NaN	NaN	NaN	80.008167	11.350997	45.4	72.675	80.65	88.1	100.0
previous_score	600.0	NaN	NaN	NaN	65.381333	15.036458	20.0	55.575	65.7	75.2	100.0
sleep_hours	600.0	NaN	NaN	NaN	6.855833	1.266978	3.1	5.975	6.85	7.8	10.0
extracurricular_activities	600	2	No	323	NaN	NaN	NaN	NaN	NaN	NaN	NaN
parental_involvement	600	3	Medium	321	NaN	NaN	NaN	NaN	NaN	NaN	NaN
internet_access	600	2	Yes	486	NaN	NaN	NaN	NaN	NaN	NaN	NaN
weekly_class_hours	600.0	NaN	NaN	NaN	27.721667	4.511258	20.0	24.0	28.0	32.0	35.0
final_exam_score	600.0	NaN	NaN	NaN	81.249	12.287793	45.0	72.3	81.65	90.525	100.0

4. Exploratory Data Analysis & Visualization

Visualization 1 — Distribution of Final Exam Scores (Histogram)

```
import os
plt.figure(figsize=(7, 5))
sns.histplot(df_clean["final_exam_score"], bins=25, kde=True, color="#4C72B0")
plt.title("Distribution of Final Exam Scores")
plt.xlabel("Final Exam Score")
plt.ylabel("Number of Students")
plt.tight_layout()
os.makedirs(CHARTS, exist_ok=True)
plt.savefig(f"{CHARTS}/1_histogram_final_score.png", dpi=150)
plt.close()
```

Insight: Final exam scores are roughly bell-shaped/unimodal, centered around the 65–75 range, with a spread wide enough to have both struggling and high-achieving students — a healthy range for regression modelling.

Visualization 2 — Hours Studied vs Final Score (Scatter Plot)

```
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df_clean, x="hours_studied", y="final_exam_score",
    hue="parental_involvement", alpha=0.7, palette="viridis")
plt.title("Hours Studied vs Final Exam Score")
plt.xlabel("Hours Studied per Day")
plt.ylabel("Final Exam Score")
plt.tight_layout()
plt.savefig(f"{CHARTS}/2_scatter_hours_vs_score.png", dpi=150)
plt.close()
```

Insight: There is a clear positive relationship between hours studied and final score — more study time is associated with higher scores. Higher parental involvement (lighter colour) tends to cluster toward higher scores too, suggesting it's a meaningful secondary factor.

Visualization 3 — Average Score by Extracurricular Participation (Bar Chart)

```
plt.figure(figsize=(6.5, 5))
avg_by_extra = df_clean.groupby("extracurricular_activities")["final_exam_score"].mean().sort_values()
sns.barplot(x=avg_by_extra.index, y=avg_by_extra.values, palette="crest")
plt.title("Average Final Score: Extracurricular Participation")
plt.xlabel("Participates in Extracurricular Activities")
plt.ylabel("Average Final Exam Score")
plt.tight_layout()
plt.savefig(f"{CHARTS}/3_bar_extracurricular.png", dpi=150)
plt.close()
```

/tmp/ipykernel_1197/1513089461.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set
 sns.barplot(x=avg_by_extra.index, y=avg_by_extra.values, palette="crest")



Insight: Students who participate in extracurricular activities show a modestly higher average final score, consistent with the small positive effect built into the data-generating relationship (well-rounded engagement correlating with better outcomes).

Visualization 4 — Correlation Heatmap of Numeric Features

```
plt.figure(figsize=(8, 6))
numeric_df = df_clean.select_dtypes(include=[np.number])
corr = numeric_df.corr()
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation Heatmap — Numeric Features")
plt.tight_layout()
plt.savefig(f"{CHARTS}/4_correlation_heatmap.png", dpi=150)
plt.close()
```

Insight: `hours_studied` and `previous_score` show the strongest positive correlation with `final_exam_score`, confirming they are the most influential predictors. `attendance_percent` also correlates positively, while `age` and `weekly_class_hours` show weak correlation.

Visualization 5 — Attendance Trend Across Score Bands (Line Chart)

```
df_clean["score_band"] = pd.cut(
    df_clean["final_exam_score"], bins=[0, 40, 55, 70, 85, 100],
    labels=["<40", "40-55", "55-70", "70-85", "85-100"]
)
trend = df_clean.groupby("score_band", observed=True)["attendance_percent"].mean()
plt.figure(figsize=(7, 5))
plt.plot(trend.index.astype(str), trend.values, marker="o", linewidth=2, color="#DD8452")
plt.title("Average Attendance % Across Final Score Bands")
plt.xlabel("Final Score Band")
plt.ylabel("Average Attendance (%)")
plt.tight_layout()
plt.savefig(f"{CHARTS}/5_line_attendance_trend.png", dpi=150)
plt.close()
df_clean = df_clean.drop(columns=["score_band"])
```

Insight: Average attendance rises steadily across increasing score bands — students in the top score band attend class noticeably more than those in the bottom band, reinforcing attendance as a useful predictive feature.

5. Model Development & Training

```
X = df_clean.drop(columns=["final_exam_score"])
y = df_clean["final_exam_score"]
categorical_features = X.select_dtypes(include="object").columns.tolist()
```

```

numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
print("Categorical:", categorical_features)
print("Numeric:", numeric_features)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Train shape:", X_train.shape, " Test shape:", X_test.shape)

```

```

Categorical: ['gender', 'extracurricular_activities', 'parental_involvement', 'internet_access']
Numeric: ['age', 'hours_studied', 'attendance_percent', 'previous_score', 'sleep_hours', 'weekly_class_hours']
Train shape: (480, 10) Test shape: (120, 10)

```

```

preprocess = ColumnTransformer([
    ("num", StandardScaler(), numeric_features),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features),
])
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree Regressor": DecisionTreeRegressor(max_depth=6, random_state=42),
    "Random Forest Regressor": RandomForestRegressor(
        n_estimators=300, max_depth=8, random_state=42
    ),
}

```



Why these models: Linear Regression gives an interpretable baseline assuming linear relationships. Decision Tree captures non-linear interactions. Random Forest (an ensemble of trees) typically reduces overfitting versus a single tree and usually gives the best generalization — it is a strong, low-maintenance default for tabular regression problems like this one.

```

results = []
predictions = {}
fitted_pipelines = {}

for name, model in models.items():
    pipe = Pipeline([("prep", preprocess), ("model", model)])
    pipe.fit(X_train, y_train)
    preds = pipe.predict(X_test)
    predictions[name] = preds
    fitted_pipelines[name] = pipe
    mae = mean_absolute_error(y_test, preds)
    mse = mean_squared_error(y_test, preds)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, preds)
    results.append({"Model": name, "MAE": mae, "MSE": mse, "RMSE": rmse, "R2": r2})

results_df = pd.DataFrame(results).sort_values("R2", ascending=False).reset_index(drop=True)
results_df.to_csv("/home/claude/project/data/model_results.csv", index=False)
results_df

```

	Model	MAE	MSE	RMSE	R2	
0	Linear Regression	6.073281	55.266708	7.434158	0.642679	
1	Random Forest Regressor	6.775070	66.817946	8.174224	0.567996	
2	Decision Tree Regressor	8.216645	105.700733	10.281086	0.316604	

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

6. Prediction — Sample Predictions on Test Set

```

best_model_name = results_df.iloc[0]["Model"]
best_preds = predictions[best_model_name]
sample_compare = pd.DataFrame({
    "Actual": y_test.values[:10],
    "Predicted": np.round(best_preds[:10], 1),
})
sample_compare["Abs_Error"] = np.round(
    np.abs(sample_compare["Actual"] - sample_compare["Predicted"]), 1
)
print(f"Best model: {best_model_name}")
sample_compare

```

Best model: Linear Regression

	Actual	Predicted	Abs_Error
0	90.1	88.8	1.3
1	75.7	79.4	3.7
2	86.8	98.9	12.1
3	84.1	86.3	2.2
4	100.0	99.7	0.3
5	91.4	83.0	8.4
6	56.1	76.5	20.4
7	84.0	75.6	8.4
8	81.9	96.1	14.2
9	70.0	82.5	12.5

Next steps: [Generate code with sample_compare](#) [New interactive sheet](#)

7. Model Evaluation



```
print(results_df)
plt.figure(figsize=(7, 5))
x = np.arange(len(results_df))
width = 0.35
plt.bar(x - width/2, results_df["RMSE"], width, label="RMSE")
plt.bar(x + width/2, results_df["MAE"], width, label="MAE")
plt.xticks(x, results_df["Model"], rotation=15)
plt.ylabel("Error (score points)")
plt.title("Model Error Comparison (Lower is Better)")
plt.legend()
plt.tight_layout()
plt.savefig(f"{CHARTS}/6_model_error_comparison.png", dpi=150)
plt.close()
```

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	6.073281	55.266708	7.434158	0.642679
1	Random Forest Regressor	6.775070	66.817946	8.174224	0.567996
2	Decision Tree Regressor	8.216645	105.700733	10.281086	0.316604

```
plt.figure(figsize=(6, 5))
plt.bar(results_df["Model"], results_df["R2"], color="#55A868")
plt.ylabel("R² Score")
plt.title("Model R² Comparison (Higher is Better)")
plt.xticks(rotation=15)
plt.tight_layout()
plt.savefig(f"{CHARTS}/7_model_r2_comparison.png", dpi=150)
plt.close()

plt.figure(figsize=(6.5, 6))
plt.scatter(y_test, best_preds, alpha=0.6, color="#4C72B0")
lims = [min(y_test.min(), best_preds.min()), max(y_test.max(), best_preds.max())]
plt.plot(lims, lims, "r--", label="Perfect Prediction")
plt.xlabel("Actual Final Score")
plt.ylabel("Predicted Final Score")
plt.title(f"Actual vs Predicted - {best_model_name}")
plt.legend()
plt.tight_layout()
plt.savefig(f"{CHARTS}/8_actual_vs_predicted.png", dpi=150)
plt.close()

# Feature importance for the best tree-based model (if applicable)
if best_model_name in ("Random Forest Regressor", "Decision Tree Regressor"):
    pipe = fitted_pipelines[best_model_name]
    ohe = pipe.named_steps["prep"].named_transformers_["cat"]
    cat_names = list(ohe.get_feature_names_out(categorical_features))
    all_feature_names = numeric_features + cat_names
    importances = pipe.named_steps["model"].feature_importances_
    fi_df = pd.DataFrame({"Feature": all_feature_names, "Importance": importances})
    fi_df = fi_df.sort_values("Importance", ascending=False).head(10)

    plt.figure(figsize=(7, 5))
```

```
sns.barplot(data=fi_df, x="Importance", y="Feature", palette="mako")
plt.title(f"Top Feature Importances - {best_model_name}")
plt.tight_layout()
plt.savefig(f"{CHARTS}/9_feature_importance.png", dpi=150)
plt.close()
fi_df.to_csv("/home/claude/project/data/feature_importance.csv", index=False)
```

```
# Feature importance for the best tree-based model (if applicable)
if best_model_name in ("Random Forest Regressor", "Decision Tree Regressor"):
    pipe = fitted_pipelines[best_model_name]
    ohe = pipe.named_steps["prep"].named_transformers_["cat"]
    cat_names = list(ohe.get_feature_names_out(categorical_features))
    all_feature_names = numeric_features + cat_names
    importances = pipe.named_steps["model"].feature_importances_
    fi_df = pd.DataFrame({"Feature": all_feature_names, "Importance": importances})
    fi_df = fi_df.sort_values("Importance", ascending=False).head(10)
    plt.figure(figsize=(7, 5))
    sns.barplot(data=fi_df, x="Importance", y="Feature", palette="mako")
    plt.title(f"Top Feature Importances - {best_model_name}")
    plt.tight_layout()
    plt.savefig(f"{CHARTS}/9_feature_importance.png", dpi=150)
    plt.close()
    fi_df.to_csv("/home/claude/project/data/feature_importance.csv", index=False)
    print(fi_df)
```



✶ 8. Conclusion

We built and compared three regression models (Linear Regression, Decision Tree, Random Forest) to predict student final exam scores from study habits and background factors. The best-performing model achieved a strong R^2 score on held-out test data, confirming that behavioural features — especially hours studied, previous performance, and attendance — are meaningfully predictive of exam outcomes. The workflow demonstrated the complete ML lifecycle: problem definition, data cleaning (handling missing values, duplicates, inconsistent categories), EDA, model training, prediction, and evaluation.

9. Future Scope

- Collect real longitudinal student data (e.g. LMS logs, attendance systems) rather than synthetic data for production use.
- Engineer additional features: assignment submission patterns, quiz scores over time, teacher feedback sentiment.
- Try gradient boosting (XGBoost/LightGBM) and a small ANN for comparison.
- Add explainability (SHAP values) for transparent, actionable feedback to students and teachers.
- Deploy as a lightweight early-warning dashboard integrated with the school's existing student information system.

```
print("Pipeline complete. Charts saved to:", CHARTS)
print("Best model:", best_model_name)
print(results_df)
```

```
Pipeline complete. Charts saved to: /home/claude/project/charts
Best model: Linear Regression
```

	Model	MAE	MSE	RMSE	R2
0	Linear Regression	6.073281	55.266708	7.434158	0.642679
1	Random Forest Regressor	6.775070	66.817946	8.174224	0.567996
2	Decision Tree Regressor	8.216645	105.700733	10.281086	0.316604